

Introduction

A broad definition: "In a general way, we can define computing to mean any goal-oriented activity requiring, benefiting from, or creating computers."

More Formal: "Type of arithmetic or non-arithmetic calculation that is well-defined."

- Computers have become ubiquitous (can find them everywhere).
- Varied usage: Social media, ChatGPT, weather forecast, computer-aided design etc.
- A lot of compute happens on the 'Cloud' (Not your computer but that of GAFAM mostly).

History

Modern computing started with a desire for a precise language for philosophical questions.

- May have heard of Turing (Imitation Game) but might not have heard of Godel.
- Punching card machines.
- Basic electronics with limited functionality.
- General purpose computers.
- Artificial Intelligence (AI) is the newest frontier.
- But there are theoretical limits! Uncomputable problems exist.

Practical computers

It takes two for a tango. Need both **Hardware** and **Software**. Hardware involves actual physical machines which perform the operations that are involved in computing. Software just specifies what is to be done which is then handled by the hardware. In software, we define a set of actions that are performed for a desired result.

- Abacus with beads and stones.
- Mechanical: Pascaline (Pascal), Difference Engine (Babbage).
- Electronic: From Vacuum tubes to transistors.

Modern day computers: Behind the scenes

Behind the scenes, the modern computers use transistors whose voltage has two levels. On/Off, 1/0, True/False. Called Boolean honouring George Boole. Information in computers is represented using building blocks called bits. These can take only two values zero or one. The basic idea is to put a lots of bits together to represent everything (a bit exaggerated). What can we represent using bits?

- Human speech, human language, animal sounds, assignment submissions.
- Concepts in human thought using AI.
- Images, Videos.
- and a lot more.

The mathematics behind zeroes and ones

We start with the decimal numbers we know well. You might see on your apps something like this:

```
likes = 3456
```

What does this mean. You have three thousand, four hundred, fifty, and six likes.

```
likes = 3x1000 + 4x100 + 5x10 + 6
```

We are so used to the decimal system that we do not think of them composed in this manner. This will change when dealing with computers because it only deals with ones and zeros. We don't have the numbers 2 to 9.

The decimal system is actually a **Base** 10 system this means that we represent using ten symbols and powers of ten. Going back:

$$likes = 3 \times 10^3 + 4 \times 10^2 + 5 \times 10^1 + 6 \times 10^0$$

If we use only two symbols it is called binary system, if eight it is called Octal (Latin inspired?), and if sixteen it is called Hexadecimal (six plus ten basically).

Example: Binary System

$$\begin{aligned} \text{students} &= 264 \\ \text{students} &= 256 + 8 \\ \text{students} &= 1 \times 2^8 + 0 \times 2^7 + \dots + 0 \times 2^0 \\ \text{students} &= (100001000)_2 \end{aligned}$$

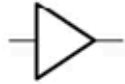
Questions

- Can we have a base 1 system? How will it look? Remember prisoners counting days in films.
- Given $a_3 \times 2^3 + a_2 \times 2^2 + a_1 \times 2^1 + a_0 = 15$, How do you find the value of a_0 ? **Hint:** Think of the divisor, dividend, quotient, and remainder when you perform $15 \div 2$.

How do computers do this math?

Since computers only can understand bits they have to perform math using operations on bits. This is accomplished using basic logic gates and binary logic.

YES



INPUT		OUTPUT
A		
0		0
1		1

NOT



INPUT		OUTPUT
A		
0		1
1		0

AND



INPUT		OUTPUT
A	B	
0	0	0
1	0	0
0	1	0
1	1	1

OR



INPUT		OUTPUT
A	B	
0	0	0
1	0	1
0	1	1
1	1	1

XOR



INPUT		OUTPUT
A	B	
0	0	0
1	0	1
0	1	1
1	1	0

NAND



INPUT		OUTPUT
A	B	
0	0	1
1	0	1
0	1	1
1	1	0

NOR



INPUT		OUTPUT
A	B	
0	0	1
1	0	0
0	1	0
1	1	0

XNOR



INPUT		OUTPUT
A	B	
0	0	1
1	0	0
0	1	0
1	1	1

We have already seen that in the binary system we put together bits in a sequence to represent numbers.

- We have *short*, *int* and *long* integers depending on the number of bits they contain. Traditionally,
- *short* has 16 bits
- *int* has 32 bits
- *long* has 64 bits

We have only seen binary logic operations on bits but how do we do the usual math like addition, subtraction, multiplication, and division?

The answer is that we have to use a combination of binary logic operations. For binary addition without carry we use **XOR** operation and for just getting the carry term we use **AND**. See below example,

```
15 ->      1 1 1 1
10 ->      1 0 1 0
carry ->    1 1 1 - -
-----
25 ->      1 1 0 0 1
```

Programming

We will be learning how to write programs. Programs are a set of instructions we give the computer. They tell them what to do. We need a language in which to write a program.

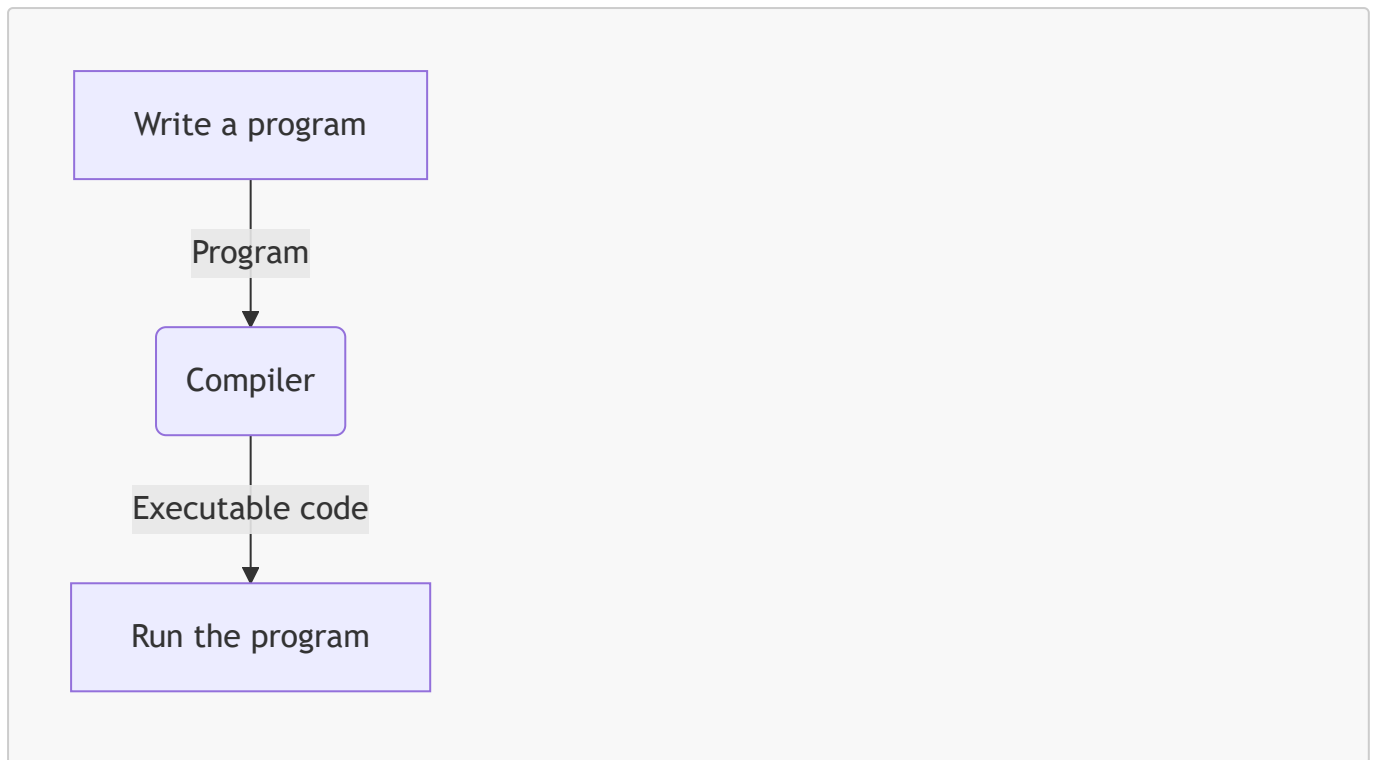
- High-level languages: C/C++, Java, Python, Javascript etc.
- High-level languages are human readable.
- But, computers only understand binary. Compiler!
- Low-level languages: Machine code (binary code), Assembly language.



Three steps of programming

The programming language C is a *compiled* high-level language. Therefore, we need to follow the following steps:

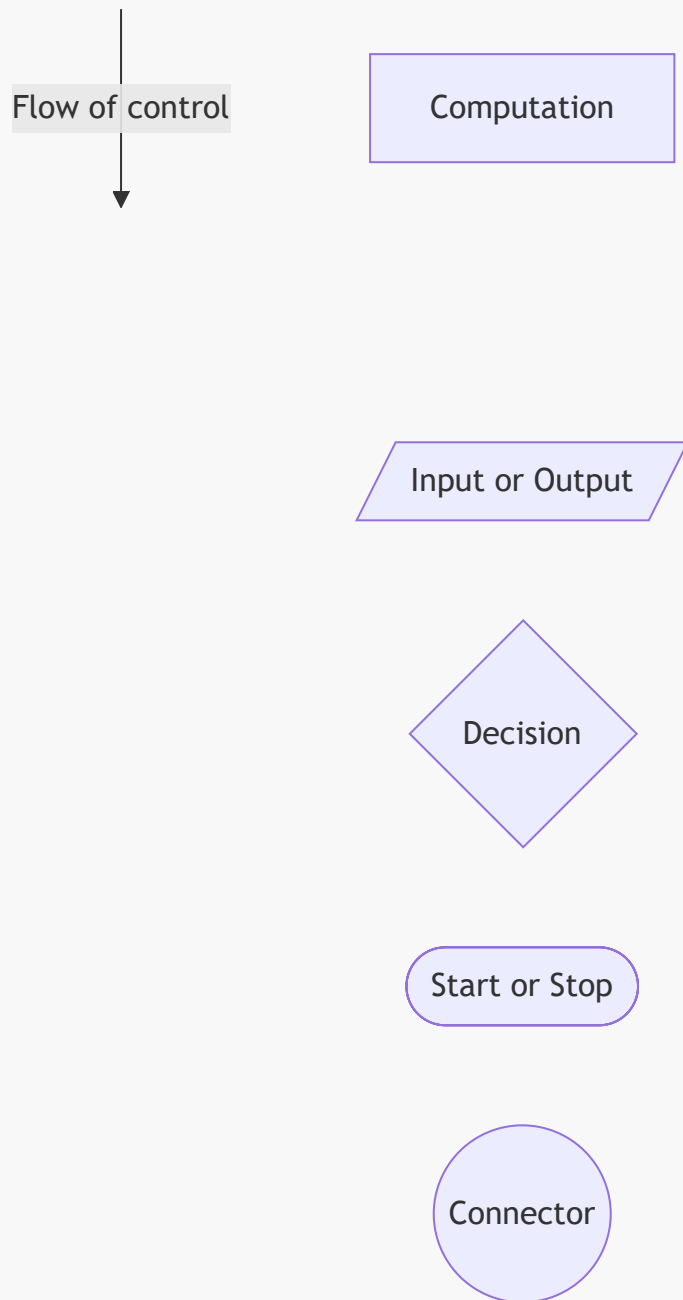
- Write the program in a high-level language.
- Compile the program using a compiler.
- Get the executable code which is binary.
- Run the program.



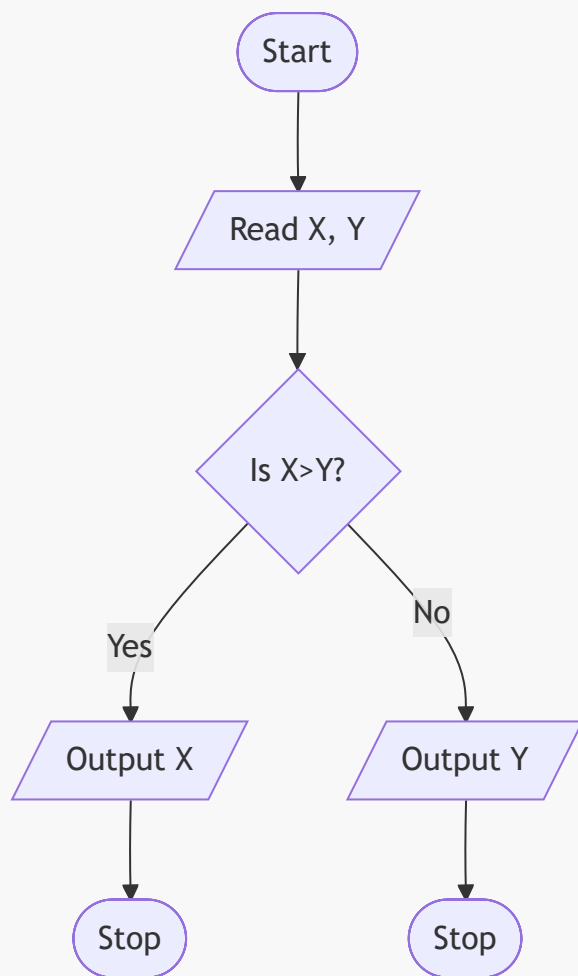
There are some programming languages that do not need compilation. They can be directly run using an *interpreter*. They are called *interpreted* languages. The interpreter reads our code and executes it for us. Famous example is *Python*.

Flowcharts

You can also think of programs in terms of flowcharts.



Simple Example



What is this flowchart computing?

A sample C program

```
/* Multiline comment
   Describe the code
*/

#include <stdio.h>

int main(){
    // Single line Comment
    printf("Hello world!\n");
    return 0;
}
```



The image shows a code editor window with a file named `main.c`. The code is as follows:

```
1  /* Multiline comment
2     Describe the code
3  */
4
5  #include <stdio.h>
6
7  int main(){
8      // Single Line Comment
9      printf("Hello world!\n");
10     return 0;
11 }
```

Below the code editor, there is a terminal window showing the output of the program:

```
Hello world!

...Program finished with exit code 0
Press ENTER to exit console.
```

Things to note:

- What is the *exit code*?
- You can include *libraries* that are not written by you.
- You will generally only use a limited number of standard libraries in this course.
- Main is the *entry point* of any program.

References

<https://cs.calvin.edu/activities/books/processing/text/01computing.pdf>

[Binary numbers and addition](#)

<https://www3.ntu.edu.sg/home/ehchua/programming/java/datarepresentation.html>